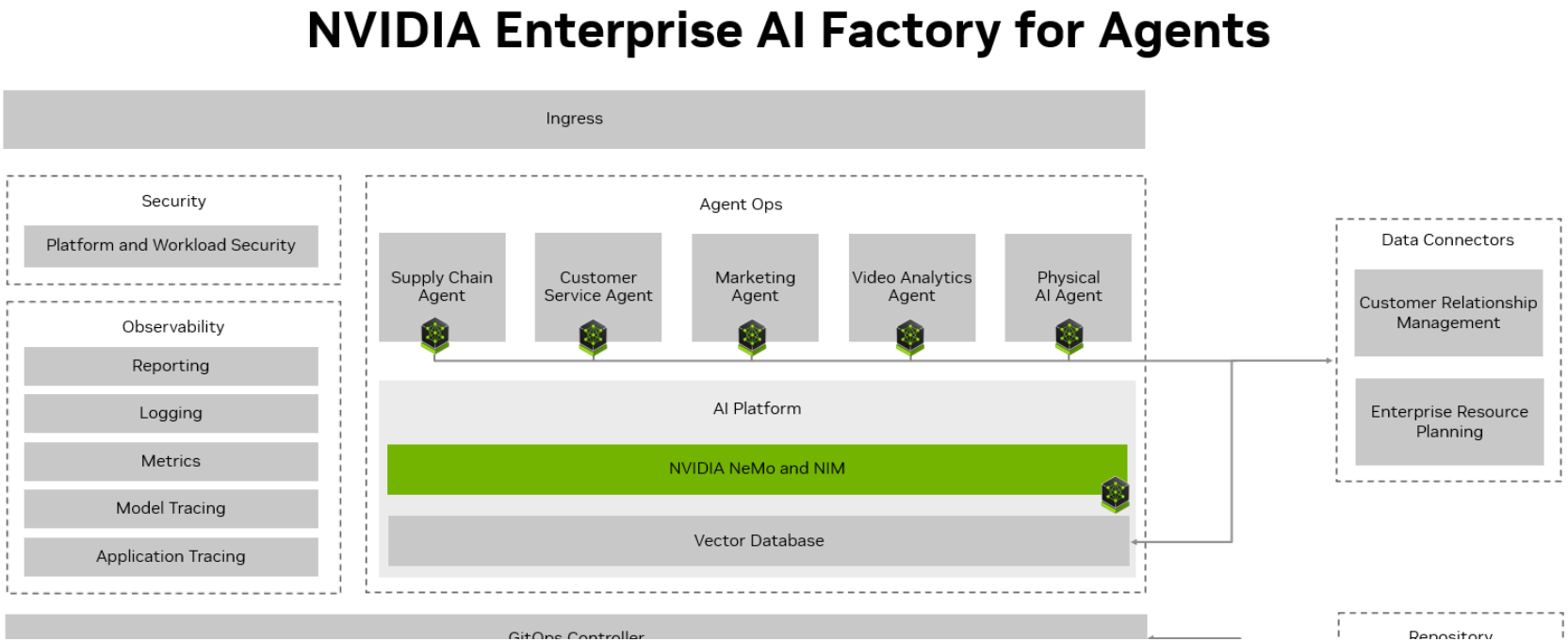
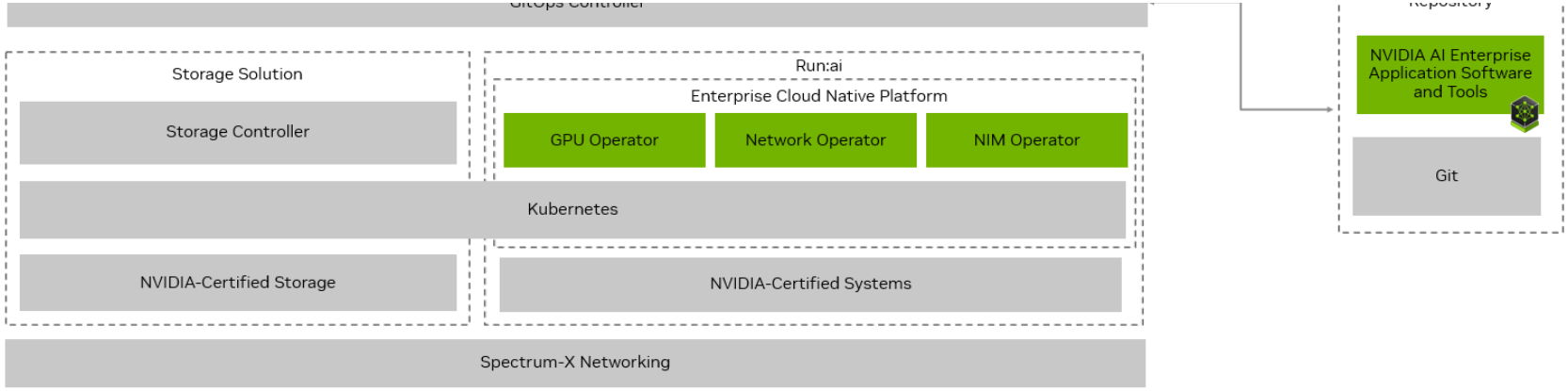
docs.nvidia.com

# Agentic AI in the Factory — NVIDIA Enterprise AI Factory Design Guide White Paper

24–30 minutes

The following sections describe the implementation of Agentic AI within an Enterprise AI Factory. Information is provided for each of the Enterprise AI Factory components (AI Platform, Data Connectors, Artifact Repository, Observability, Security, & Hardware) in relation to Agentic AI.






## Enterprise Cloud Native Platform#

NVIDIA-Certified systems offer a reliable platform for deploying high-performance accelerated software, including NVIDIA AI Enterprise. They ensure consistent performance and allow enterprises to deploy optimized platforms for AI, Data Analytics, HPC, high-density VDI, and other data center workloads.

The Enterprise Cloud Native Platform, with Kubernetes at its core, provides agility, scalability, and resilience for an Enterprise AI Factory focused on developing and deploying sophisticated AI agents. Kubernetes embodies cloud-native principles by orchestrating containers (like those from NVIDIA AI Enterprise), managing microservice-based agent architectures, and enabling dynamic automation. This includes automated deployment of new agent versions, scaling based on demand (important for both training and inference on NVIDIA-Certified Systems), self-healing to support high availability, and resource management, particularly for GPU resources.

These cloud-native capabilities are relevant for an AI Factory. The ability to independently develop, update, and scale microservice-based agents, coupled with automated CI/CD pipelines managed via Kubernetes, allows for iteration and deployment. Kubernetes handles the significant and often

burstable compute demands for training AI models and scales inference services for deployed agents based on real-time needs. This automation and resource packing on NVIDIA-Certified Systems also contribute to reducing operational burden and optimizing costs, which is a consideration when dealing with complex AI/ML environments, especially those involving GPUs.

In this context, Kubernetes functions as a foundational platform for the AI Factory. It unifies the management of a complex stack—including NVIDIA Operators, AI software suites like NVIDIA AI Enterprise, storage, networking, and observability tools—onto a single platform. Enterprise Kubernetes distributions and validated architectures can further simplify this by providing secure, supported, and pre-integrated environments. This orchestration supports the process of efficiently building, deploying, and managing a diverse and evolving suite of AI agents on high-performance infrastructure.

## Storage Solution#

A key component of the Enterprise AI Factory is its storage solution. Given the continuous and high-volume data flow required throughout the AI development and deployment lifecycle, the storage infrastructure can potentially become a significant bottleneck if not architected correctly to meet these intensive demands. Therefore, the solution must possess several essential features to handle demanding AI environments. These include the scalability to manage exponentially growing datasets and model sizes, and the flexibility to support diverse data types and access patterns, ranging from high-throughput sequential reads for training to low-latency random access for inference and vector databases. Furthermore, robust data protection mechanisms like snapshots, replication, and disaster recovery are critical, as are comprehensive security features such as encryption at rest and in transit to safeguard sensitive information.

To meet these diverse workload requirements, the Enterprise AI Factory's storage solution utilizes a

tiered storage architecture from various vendors. A crucial element of this architecture is NVIDIA-Certified Storage, which adheres to stringent performance and reliability standards specifically for AI tasks. This certification ensures efficient data access, which is vital for handling large model weights, managing Vector Database I/O for Retrieval Augmented Generation (RAG), and supporting knowledgebases for AI Agents. Having been vetted for these crucial characteristics, the certified storage provides a dependable, high-performance, secure, and scalable infrastructure, thereby enhancing the overall efficiency and stability of the AI Factory. This empowers partners and customers to build AI factories that efficiently leverage massive amounts of data, leading to faster, more accurate, and reliable AI models.

The NVIDIA-Certified Storage program offers two levels of certification: Foundation and Enterprise. These storage certifications integrate seamlessly with corresponding NVIDIA Enterprise RAs to ensure that storage systems possess the necessary performance to support North-South networking and effectively feed data to compute nodes. The Foundation-level storage certification certifies storage partners for PCIe-optimized reference configurations specifically for the NVIDIA RTX PRO 6000 Blackwell Server Edition. The larger-scale Enterprise-level storage certification validates storage partners for HGX reference configurations, particularly for the NVIDIA HGX B200.

## Artifact Repository#

The AI Factory incorporates a dedicated artifact repository designed to handle software components, especially for on-premises setups that follow GitOps principles. This repository serves as a secure, version-controlled local hub for essential NVIDIA AI Enterprise artifacts, such as containerized NVIDIA NIM microservices, AI models, libraries, and tools. In the GitOps workflow, git maintains the declarative state by linking to specific versions of these NVIDIA artifacts stored in the repository. Then, the GitOps Controller fetches these verified artifacts for deployment onto the Kubernetes platform. For on-

premises environments, managing local artifacts enables essential operational practices such as scanning NVIDIA containers and other artifacts for security vulnerabilities, ensuring reliable and rapid access without relying on public registries, managing dependencies, and assuring reproducible deployments using specific, approved versions of NIMs and AI models.

## GitOps Controller#

A GitOps controller is a software component that continuously monitors the desired state of infrastructure and application configurations stored in a Git repository and ensures that the actual state of a system, such as a Kubernetes cluster, matches this declared state. Working in close collaboration with the artifact repository, it operates by regularly comparing the live state of resources in the environment with the version-controlled configurations in Git. If any differences are detected, the controller automatically reconciles them by applying the necessary changes to bring the system back in sync with what is defined in the repository. This approach leverages Git as the single source of truth, enabling automated, auditable, and reproducible deployments, and is typically implemented as a Kubernetes controller that runs a reconciliation loop to maintain consistency between Git and the cluster.

## Observability#

Ensuring the reliability, performance, and trustworthiness of an AI Agent Platform requires a robust observability strategy. Observability provides deep insights into the system's state and behavior, enabling teams to proactively identify issues, debug complex interactions, and optimize performance. This is achieved through a combination of comprehensive logging, continuous monitoring of key metrics, detailed model tracing, thorough application tracing (often as part of distributed tracing), and consolidated reporting to understand the flow of operations and overall health of the platform.

Centralized logging forms a foundational aspect of this strategy, capturing detailed events from all layers of the platform. This includes logs from the underlying infrastructure, the container platform, core AI software components, and the AI agents themselves. These logs are invaluable, offering heuristics for debugging agent behavior, supporting security analysis by providing event records, and creating comprehensive audit trails. Such trails are essential for ensuring operational reliability, building trust in the AI system's outputs, and meeting compliance requirements.

Continuous monitoring of metrics is implemented to track both the health of the infrastructure and critical Key Performance Indicators (KPIs) specific to the AI agents. Key metrics, often collected using OpenTelemetry (OTEL) instrumentation, Application Performance Management (APM) tools, or directly from application endpoints and infrastructure tools or NVIDIA Data Center GPU Manager (DCGM), provide a real-time view of system performance and agent effectiveness. These metrics typically fall into the following categories:

- **Latency**:
- *Time To First Token (TTFT)*: The delay before the agent produces its initial response token after receiving a request.
- *Tokens Per Second (TPS) / Output Throughput*: The rate at which the agent generates response tokens over time.
- *End-to-End Latency*: The total time elapsed from the user's request to the completion of the agent's full response.
- *Component Latency*: The duration of individual processing steps, such as:
- Plan Generation
- Reasoning

- Tool Calls
- Database Queries (including vector database lookups)
- Retriever Calls
- **Accuracy and Faithfulness**:
- *Task Completion Rate*: The percentage of assigned tasks that the agent successfully completes.
- *Accuracy/Relevance*: The correctness and relevance of responses that rely on retrieved information (RAG), with specific metrics for retriever performance including precision, recall, and F1-score.
- *Faithfulness*: How well the agent's responses adhere to the provided source information, particularly in RAG scenarios.
- *Correctness*: The validity of outputs from individual reasoning steps or executed tools.
- **Resource Utilization**:
- Consumption of GPU, CPU, and memory during agent operation.
- **Errors and Faults**:
- *Fault Rate*: The frequency of errors or failures within specific agent components or workflows (e.g., plan generation, tool calls, database access).
- *Timeout Rate*: The number of operations that exceed their allocated time limit, categorized by component (e.g., tool call timeouts, retriever timeouts).

To understand operational flows and decision-making pathways, detailed model and application tracing is important. This involves mapping the journey of requests and the sequence of actions across the AI Agent Platform's distributed services. This includes tracing interactions between agents, RAG components (specifically retriever calls), various database calls, inference endpoints, and individual

tool calls an agent makes. Contemporary tracing methods, often facilitated by tools like OpenTelemetry (OTEL) and visualized in Application Performance Management (APM) systems, should capture the inputs, outputs, and duration of an agent's plan generation step, as well as the logical processes in each reasoning step, offering insight into intermediate processes where feasible. This level of detailed tracing is useful for identifying performance bottlenecks, pinpointing latency sources within complex agent interactions—such as those arising from specific tool, database, or retriever calls—and providing important context by correlating traces with other metrics and logs.

To make the collected observability data useful, a consolidated reporting mechanism is beneficial. This involves presenting aggregated data from logging, metrics, and tracing in dashboards and reports. Such presentations offer a view of the AI Agent Platform's health, performance, and accuracy, which can be tailored for different user roles, including IT operations, AI developers, and business stakeholders.

By extending the observability focus to these detailed aspects of AI agent operations—such as the specifics of planning and reasoning steps, the performance and accuracy of tool and data retrieval calls, and detailed fault and timeout analysis—enterprises can develop a more thorough understanding and improve control over their AI solutions. This increased visibility contributes to building and maintaining AI systems that are powerful, robust, efficient, and trustworthy.

## Security#

A robust security posture for the platform is achieved through a multi-layered strategy. This approach uses core component capabilities and integrates with enterprise security frameworks to safeguard operations and data comprehensively, from the network perimeter down to individual data elements.

The first line of defense is established at the network level. This layer employs defense-in-depth

strategies, primarily utilizing network policies native to the underlying container orchestration platform. These policies control traffic flow between services and pods, isolating workloads and restricting communication to only authorized pathways, thereby minimizing the attack surface. To further bolster this, dedicated communication infrastructure enforces fine-grained traffic control policies at the application layer and automatically encrypts all traffic between services, ensuring secure and authenticated communication channels throughout the platform.

Building upon network controls, this layer focuses on verifying user and service identities and their entitlements. Authentication and authorization mechanisms are typically integrated directly with the platform's own access systems. Crucially, these are seamlessly tied to broader enterprise Identity and Access Management (IAM) solutions, such as corporate directory services. This ensures consistent identity management and allows for centralized control over user access based on established enterprise credentials and policies.

Once identity is established, access to platform resources is governed by Role-Based Access Control (RBAC). This is implemented at multiple levels:

- **Orchestration Platform RBAC**: The container orchestration platform itself employs RBAC to control permissions for managing and interacting with cluster resources (e.g., deploying applications, accessing logs, configuring services).
- **Integrated Platform RBAC**: AI/ML platforms integrated within the ecosystem also commonly feature their own RBAC systems. These ensure that access to platform-specific functionalities and resources is restricted based on predefined user or service roles.

The most granular layer of security focuses on protecting the data itself within specialized data services, including various database systems. These services are often further protected by their internal RBAC mechanisms. These controls manage fine-grained access to data elements—such as

specific data sets, tables, or collections—ensuring that read and write permissions are granted exclusively to authenticated and appropriately authorized applications and users, adhering to the principle of least privilege.

Clear roles and responsibilities are defined for effective management and operation of the complex on-premise, self-hosted AI/ML platform. This document outlines a consolidated mapping of key enterprise organizational roles to their typical access levels or administrative duties within logical groupings of platform tools. The goal is to provide a concise overview that facilitates understanding of how different teams interact with the various components of the technology stack, ensuring security, efficiency, and clear accountability in a self-managed environment.

| | **IT Admin** | **Network Administrator** | **AI Developer** | **MLOps** |
|---|---|---|---|---|
| **Enterprise cloud native platform** (e.g., Kubernetes/ OpenShift, Base OS, Compute/ GPU Management) | **Platform Admin & Provisioning**: Manages OS, hardware, orchestration lifecycle, compute (GPUs). Ensures stability & resource availability. | **Network Infra Mgmt**: Configures platform networking (SDN, ingress, egress), physical network. Collaborates with IT Admin on cluster networking. | **Platform User**: Accesses platform for dev tools, logs, allocated compute (GPUs) for AI tasks. | **App Deployment & Ops**: Admin/ Operator in project namespaces for CI/CD agents, AI apps, services. Manages compute for inference. |

| | IT Admin | Network Administrator | AI Developer | MLOps |
|---|---|---|---|---|
| **Storage Solution** (Core & AI-specific storage, e.g., for datasets, models, VectorDBs) | **Core Storage Admin**: Manages core storage infrastructure, backups, and base provisioning. Ensures availability for platform layers. | **Network for Storage**: Ensures reliable network connectivity and segmentation for storage systems. Troubleshoots storage network issues. | **Data Consumer**: Utilizes provisioned storage for datasets, model artifacts, and vector database access. | **Storage for AI Apps**: Manages persistent storage claims for AI applications and models in production. Monitors storage performance for deployed agents. |
| **Artifact Repository** (e.g., for container images, packages, models) | **Infra Support**: Provides/ maintains underlying infra (OS, VMs, K8s) for the repository. Core infra install/ patch. | **Network Access**: Ensures repository has necessary network access and is accessible by CI/CD tools and platform. | **Artifact User/ Publisher**: Manages and versions data/ model artifacts, packages, and notebooks within the repository. Pulls base images. | **Artifact Lifecycle Mgmt**: Manages CI/CD integration for publishing and consuming application/ model artifacts and container images. |

| | IT Admin | Network Administrator | AI Developer | MLOps |
|---|---|---|---|---|
| **GitOps Controller** (e.g., ArgoCD) | **Infra Support**: Maintains underlying infra (OS, VMs, K8s) for the GitOps controller. | **Network Access**: Ensures GitOps controller can reach Git repositories and the Kubernetes API. | **User (Indirect)**: Benefits from GitOps for consistent environments defined by MLOps/ Platform teams. | **GitOps Automation Lead**: Defines/ manages application and infrastructure configurations in Git. Manages GitOps controller for deployments. |
| **Observability** (Monitoring, Logging, Tracing, Reporting) | **Infra Support**: Provides/ maintains underlying infra for the observability stack. Core infra install/patch. | **Network Support**: Ensures monitoring tools reach targets & telemetry flows to central systems. | **Dev/Experiment Monitoring**: Creates/views dashboards for experiments, data profiles, model dev metrics. Accesses logs for debugging. | **Prod AI Perf. Monitoring**: Admin/Editor for dashboards/ alerts on prod AI app performance, model drift, resource use, lifecycle quality. |

| | IT Admin | Network Administrator | AI Developer | MLOps |
|---|---|---|---|---|
| **Security** (Endpoint, Network, Identity, Data Security) | **Infra Security & IdP Support**: Manages server security tools, IdP infra. Secures base platform. Collabs on infra firewall rules. | **Network Security Impl.**: Manages firewalls, network security policies, IDS/IPS. Configures network aspects of IdP & security tools. Collabs on security posture. | **Authenticated User**: Leverages federated identity for authorized access to tools, platforms, and data. Follows security best practices. | **Secure Deployment**: Implements secure CI/CD practices. Manages secrets for deployed applications. Uses federated identity for tool/ prod access. |
| **Data Connectors** (e.g., to ERP, CRM, other enterprise systems) | **Infra & Network Support:** Ensures underlying infrastructure and network paths are available for data connectors. | **Network Connectivity**: Ensures secure and reliable network connectivity for data connectors to source/target systems. | **Data User**: Utilizes configured data connectors to ingest data for AI model development and RAG. | **Operational Monitoring**: Monitors the health and performance of data connectors used by production AI agents. |

| | IT Admin | Network Administrator | AI Developer | MLOps |
|---|---|---|---|---|
| **AI Platform** (e.g., AI/ML dev environments, Training/Fine-tuning services, Model Registries) | **Infra Support**: Provides/ maintains underlying infra (OS, K8s, GPU access) for the AI platform components. | **Network Connectivity**: Ensures AI platform components have necessary network access for data, inter-service communication, and user access. | **Primary User**: Creates projects, prepares data, builds, trains, tunes, and registers models. Uses platform tools for experimentation. | **Model Lifecycle Mgmt**: Manages the CI/CD integration for models from the AI platform to production. Monitors resource usage of platform by MLOps tools. |
| **Agent Ops** (Deployment, management, and operation of AI Agents) | **Resource Provisioning**: Ensures sufficient compute, storage, and network resources are allocated for deployed AI | **Network Services for Agents**: Configures network routes, load balancing, and access policies for AI agents. | **Agent Logic Developer**: Develops the core logic, AI model integration, and specific functionalities of the AI agents. Tests agent behavior. | **Agent Deployment & Production Mgmt**: Deploys, scales, monitors, and manages the lifecycle of AI agents in production. Implements CI/ |

| | IT Admin | Network Administrator | AI Developer | MLOps |
|---|---|---|---|---|
| | agents. | | | CD for agents. |

For secrets management, secure storage is provided using Kubernetes Secrets or specialized solutions that adhere to security procedures. Image security is reinforced by integrating container image scanning tools within artifact repositories, embedded within CI/CD pipelines. This process follows industry standard security gates to ensure the integrity of container images.

Endpoint and workload security are bolstered by real-time threat detection and response mechanisms configured in line with validated policies. For AI agent and model security, tools such as NVIDIA NeMo Guardrails and partner solutions are employed. These tools ensure input validation, output filtering, and secure execution, adhering to the best practices that have been internally validated.

Auditing is also a critical component, with comprehensive audit logging configured within associated applications. These logs are forwarded to Information and Event Management (SIEM) systems, following validated logging standards to ensure thorough and efficient monitoring of system activities. Together, these measures create a robust and secure environment to support AI workloads and platform services effectively.

A structured approach to patching and upgrades across the entire AI platform—including operating systems, container platforms, the AI software suite (e.g., NVIDIA AI Enterprise), and partner components—is crucial for security, stability, and performance. This requires rigorous testing, coordination with hardware and software vendors (leveraging the NVIDIA ecosystem and reference designs where applicable), and scheduled deployments to minimize operational disruption. Regular maintenance and updates ensure access to the latest features and security for AI agents.

## Data Connectors#

For AI agents to function effectively within the platform, they need secure access to diverse sources of enterprise data. This is achieved through connectors and API endpoints that link internal systems like customer relationship management, enterprise reporting platforms and point of sale systems. The data ingestion/retrieval system ensures security, scalability, and reliability. Ingested data is transformed into embeddings and stored in a vector database for efficient semantic searches in RAG workflows. Emerging standards, such as Model Context Protocol (MCP), aim to provide structured ways for AI agents to discover and interact with external data sources and tools. Toolkits like NVIDIA's open-source Agent Intelligence toolkit help developers build, connect and optimize the AI agents using retrieved enterprise data for complex reasoning, planning, and multi-step task execution.

## AI Platform#

An AI platform is an integrated suite of technologies that provides the infrastructure, tools, and services needed to build, train, customize, deploy, and manage machine learning and generative AI models and AI Agents at scale. Such platforms streamline the end-to-end AI development lifecycle, offering capabilities for data preparation, model training, fine-tuning, deployment, monitoring, and governance. The AI platform natively integrates frameworks like NVIDIA NeMo and NIM, enabling organizations to efficiently develop and customize large language models (LLMs) and other generative AI systems. NVIDIA NeMo provides a cloud-native, end-to-end framework for building, training, and deploying LLMs and other AI models, while NIM offers standardized microservices and APIs for seamless model deployment across cloud, on-premises, or edge environments. By leveraging these integrated tools, an AI platform empowers enterprises to create domain-specific AI solutions, accelerate innovation, and maintain secure, scalable, and high-performance AI operations.

## Agent Ops#

Enterprises can use NVIDIA AI Blueprints for accelerated development of agentic AI systems in supply chain, marketing, and customer service. These workflows combine NVIDIA NIM™ microservices with GPU components for large-scale agent deployments and include frameworks, pretrained models, Helm Charts, and Jupyter Notebooks.

The Mega Omniverse Blueprint simulates warehouse operations for supply chain optimization using physics-informed digital twins and reinforcement learning. The Digital Human Blueprint uses avatar animation, speech AI, and multimodal reasoning for virtual customer service assistants. The RAG Blueprint enhances marketing applications with a hybrid vector search and multimodal extraction pipeline for enterprise data. These Blueprints utilize NVIDIA's Agent Intelligence toolkit to connect, profile, and optimize AI agent teams across complex workflows.

## Ingress#

Ingress is a mechanism that manages and controls external access to applications and services running within private infrastructure or a cluster, such as Kubernetes. It acts as a gateway, routing HTTP and HTTPS traffic from outside the organization's network to the appropriate internal services based on configurable rules. Ingress enables features like URL-based routing, load balancing, SSL/TLS termination, and name-based virtual hosting, allowing multiple applications to be securely exposed through a single entry point. This approach simplifies network management, centralizes configuration, and enhances security by consolidating how external clients reach internal resources.